Identity Federation with Amazon Cognito

Explore how to use Amazon Cognito for identity federation to securely authenticate users using email, social providers, or corporate identities. Understand user and identity pools, IAM role integration, temporary AWS credentials, and enabling multi-factor authentication. This lesson helps you implement secure and scalable user authentication in AWS applications.

As we build applications that store personal data or offer gated functionality, authentication and authorization become critical. Amazon Cognito provides a managed identity solution that helps us securely verify who our users are and control what they can access, whether they are logging in with email, a social provider like Google, or a corporate identity through SAML.

Cognito integrates smoothly with AWS services and handles identity management tasks, allowing us to focus on building application features without worrying about token validation, credential storage, or permission scoping.

Introduction to Amazon Cognito

Amazon Cognito is a managed identity and access management service designed specifically for applications. It allows us to add user sign-up, sign-in, and access control to web and mobile apps without building authentication systems from scratch. Cognito supports authentication through usernames and passwords, third-party identity providers like Google and Facebook, as well as enterprise identity systems via SAML or OpenID Connect (OIDC). It also in with AWS Identity and Access Management (IAM) and AWS Security Token Service (STS) to enable fine-grained authorization using temporary credentials.

Ask

By using Cognito, we offload the complexity of managing user directories, tokens, and secure integration with AWS services, giving us a scalable, secure, and standards-compliant identity layer.

How Amazon Cognito works

In Amazon Cognito, we can create two types of pools: user and identity pools.

1. User pools

User pools serve as fully managed user directories that handle sign-up, sign-in, and user profile management. They support multi-factor authentication (MFA), password policies, and federation with social or enterprise identity providers.

The illustration below shows how Amazon Cognito user pools handle user authentication. This includes user registration, sign-in, and token issuance for application use.

Ask

Workflow of Cognito for authentication

We can create and configure a user pool via the AWS Console or using `boto3` in Python:

main.py Python 3.10.4

```
import boto3

client = boto3.client('cognito-idp')

response = client.create_user_pool(
    PoolName='MyAppUserPool',
    AutoVerifiedAttributes=['email']
)

user_pool_id = response['UserPool']['Id']
print("Created User Pool:", user_pool_id)
```

Ask

Create a user pool

This gives us a user pool where users can sign up using their email addresses.

Once the user pool is created, we can register users through the application frontend or programmatically:

main.py Python 3.10.4

```
import boto3

client = boto3.client('cognito-idp')

response = client.sign_up(
    ClientId='1h57kf5cpq17m0eml12EXAMPLE',
    Username='jane.doe@example.com',
    Password='MySecurePass123!',
    UserAttributes=[
        {'Name': 'email', 'Value': 'jane.doe@example.com'}
    ]
)
```

Register a user

After signing up, the user must confirm their registration, typically via a verification code sent to their email.

Once the user is registered and confirmed, they can sign in using their email and password. This process involves initiating an authentication flow using the `USER_PASSWORD_AUTH` method, which returns a set of tokens, including an ID token that represents the authenticated user. Here's how we can authenticate user using the `boto3` client:

Ask

main.py Python 3.10.4

```python
import boto3

client = boto3.client('cognito-idp')

response = client.initiate_auth(
    ClientId='1h57kf5cpq17m0eml12EXAMPLE',
    AuthFlow='USER_PASSWORD_AUTH',
    AuthParameters={
        'USERNAME': 'jane.doe@example.com',
        'PASSWORD': 'MySecurePass123!'
    }
)
```

Authenticate a user

This ID token contains claims about the user and can be used to authorize them in our application or send it to a backend for further processing.

2. Identity pools

Identity pools let us authenticate users using an external identity provider (IdP), such as Google, Facebook, or enterprise providers like Azure AD. Instead of managing passwords, we trust the IdP to verify users and return identity tokens that can be used with AWS. These tokens are exchanged for temporary credentials using Cognito identity pools and STS, following a similar process to a user pool token.

The illustration below represents the workflow for Amazon Cognito identity pools, where user identities, including those from third-party providers, are mapped to IAM roles to obtain temporary AWS credentials.

Ask

Workflow of Cognito for providing AWS account access

We can create an identity pool using the AWS Console, or use the `boto3` client for `cognito-identity`:

main.py Python 3.10.4

```
import boto3

client = boto3.client('cognito-identity')

response = client.create_identity_pool(
    IdentityPoolName='MyAppIdentityPool',
    AllowUnauthenticatedIdentities=False,
    CognitoIdentityProviders=[
        {
            'ProviderName': 'cognito-idp.us-east-1.amazonaws.com/y
            'ClientId': 'your-app-client-id',
            'ServerSideTokenCheck': True
        }
```

Ask

Create an identity pool

At this point, we have an identity pool that is aware of the user pool. However, users still cannot access AWS resources until we associate an IAM role.

To allow authenticated users to assume a role and access AWS services, we create an IAM role and link it to the identity pool:

main.py Python 3.10.4

```
import boto3

client = boto3.client('cognito-identity')

client.set_identity_pool_roles(
    IdentityPoolId='us-east-1:abc12345-d678-90ef-1234-56789abcde00',
    Roles={
        'authenticated': 'arn:aws:iam::123456789012:role/Cognito_MyApp
    }
)
```

Set a role for the identity pool

The attached IAM role must trust Cognito identity to allow `sts:AssumeRoleWithWebIdentity`.

After we've created and configured an identity pool in Amazon Cognito, we can use it to exchange user identity tokens for temporary AWS credentials. When a user signs in via a trusted identity provider, they receive an identity token (such as an ID token or access token). This token contains claims that identify the user. We pass this token to the Cognito identity pool service, which verifies it and re unique identity ID. Then, Cognito uses AWS Security Token Service (STS) under the hood to assume a preconfigured IAM role and returns temporary AWS

Ask

credentials, `AccessKeyId`, `SecretKey`, and `SessionToken`. Here's how we can program this exchange in our Python apps:

main.py Python

```
import boto3

# Identity pool details
identity_pool_id = 'us-east-1:your-identity-pool-id'
region = 'us-east-1'
id_token = 'eyJraWQiOi...your-ID-token...'

# Step 1: Create Cognito Identity client
cognito_identity = boto3.client('cognito-identity', region_name=regio

# Step 2: Retrieve the unique identity ID using the token
identity_id_response = cognito_identity.get_id(
    IdentityPoolId=identity_pool_id
```

Use identity pools to get AWS credentials

We can add the AWS credential set we receive to our application's AWS SDK client configuration to securely access AWS services on behalf of the user, based on the permissions attached to the assumed role.

IAM roles for federated users

As already discussed, when we create an identity pool in Amazon Cognito, we must specify one or more IAM roles that define what the user can do once they are authenticated. These roles can be of two types:

- **Authenticated role:** This is assigned to users who successfully sign in through an identity provider (such as a Cognito user pool, Google, Facebook or SAML).
- **Unauthenticated role:** This is optionally assigned to guests who haven't signed in, if guest access is enabled.

Ask

Each of these roles must have trust policies that allow Cognito identity to assume the role on behalf of the authenticated user. Cognito uses AWS STS under the hood to assume the role and generate temporary credentials.

We can also configure role mapping rules to assign different IAM roles based on identity provider attributes. To better understand this concept, let's walk through an example.

Suppose we have a user pool configured where authenticated users carry a `custom:role` claim that indicates their level of access—either `admin` or `user`. We can create two IAM roles:

- An `Admin` role with full S3 access
- A `User` role with read-only S3 access

In the identity pool configuration, we can enable token-based role mapping and define rules using Python SDK as shown below:

main.py Python 3.10.4

```
import boto3

client = boto3.client('cognito-identity')

client.set_identity_pool_roles(
    IdentityPoolId='us-east-1:your-identity-pool-id',
    Roles={
        'authenticated': 'arn:aws:iam::123456789012:role/Cognito_Defau
    },
    RoleMappings={
        'cognito-idp.us-east-1.amazonaws.com/us-east-1_abcde1234': {
            'Type': 'Rules',
            'AmbiguousRoleResolution': 'AuthenticatedRole'
```

Ask

Set identity pool roles

Now, when a user signs in and presents an ID token with the appropriate `custom:role` value, Cognito will use these rules to determine which IAM role to assume on behalf of the user. Based on the user's token, Cognito automatically selects the correct IAM role, assumes it using STS, and returns temporary credentials to the application.

Enabling MFA in Amazon Cognito

Multi-factor authentication (MFA) adds an extra layer of security by requiring users to provide a second verification factor, typically a time-based one-time password (TOTP) or SMS code, in addition to their username and password.

In Amazon Cognito, MFA is configured at the user pool level, and we can choose to make it optional or required.

When configuring MFA for a user pool, we can choose among the following modes:

- `OFF`: MFA is disabled for all users.
- `OPTIONAL`: Users can choose to enable MFA individually.
- `ON`: All users must complete MFA to sign in.

Here's how we can use AWS Python SDK, Boto3, to set MFA configurations:

main.py Python 3.10.4

```
import boto3

client = boto3.client('cognito-idp')

client.set_user_pool_mfa_config(
    UserPoolId='us-east-1_example',
    SmsMfaConfiguration={
        'SmsAuthenticationMessage': 'Your authentication code is {###
        'SmsConfiguration': {
            'SnsCallerArn': 'arn:aws:iam::123456789012:role/CognitoSN
            'ExternalId': 'your-external-id'
```

Ask

```
}
```

Enable MFA in Cognito

When MFA is enabled, Cognito triggers an additional challenge after the initial sign-in. Here's how it works:

1. User provides username and password.
2. Cognito returns a `ChallengeName`, such as `SMS_MFA` or `SOFTWARE_TOKEN_MFA`.
3. User responds to the challenge by entering the MFA code.
4. Cognito returns ID, access, and refresh tokens upon successful verification.

Use cases for Amazon Cognito

Some use cases of Amazon Cognito are as follows:

- **Web and mobile applications:** Cognito is ideal for adding user authentication and access control to web and mobile applications, enabling developers to focus on building core application features.
- **IoT applications:** With Cognito, IoT devices can authenticate and securely communicate with AWS services, ensuring secure access to resources and data.
- **Serverless applications:** Serverless applications can leverage Cognito for

Ask